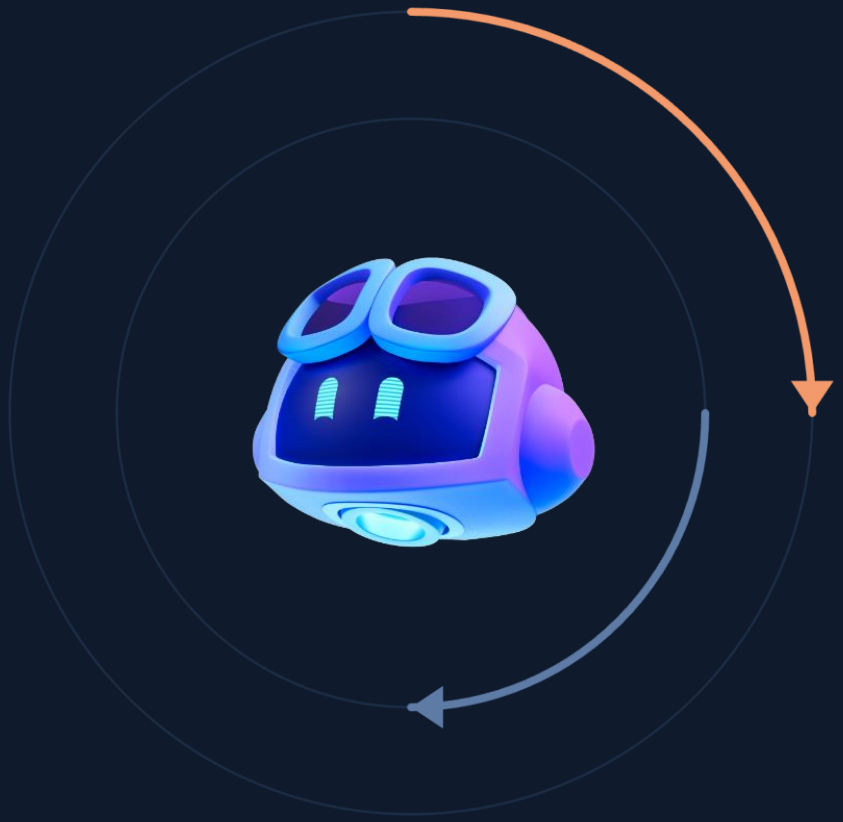


Microsoft Reactor · October 13, 2026

Continuous Agentic Modernization

From once-a-decade rewrites to
an always-on phase in the SDLC



Bruno Borges

@brunoborges · linkedin.com/in/brunocborges

AGENDA · 45 MINUTES

01

Agents in the SDLC

From autocomplete to teammates that plan, code, test and review.

02

Modernization, the old way

Why upgrading the stack was a big-bang project you survived once a decade.

03

From months to days

GitHub Copilot app modernization end to end, with a live demo.

04

Modernization as a loop

Making modernization a continuous stage of the SDLC, not a project.

01

Agents in the SDLC

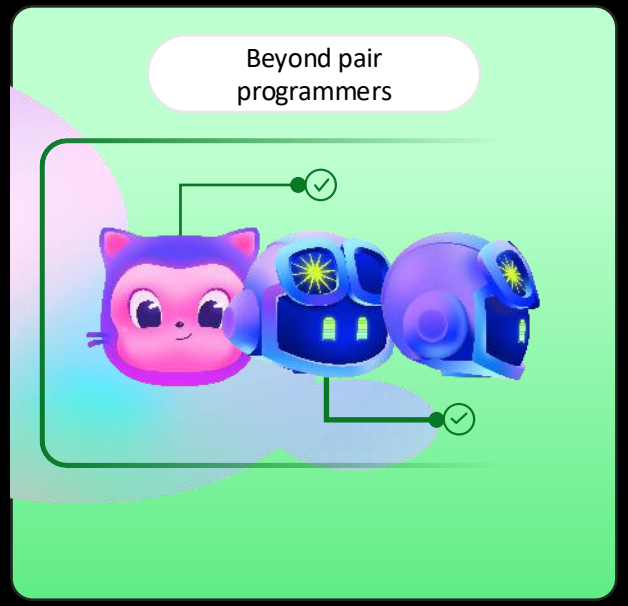
From autocomplete to teammates

Riding the waves

Wave 1 — Wave 2 — Today — Wave 3



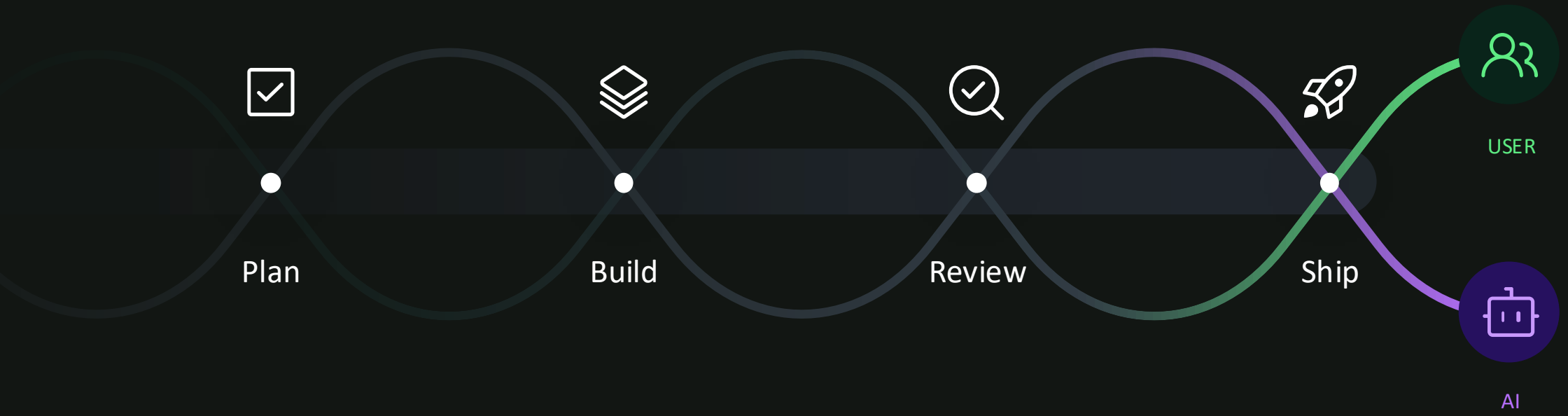
Code completion
(2021-2023)



Agentic SDLC
(2024-2025)



Hybrid teams
(2026+)


Plan
Build
Review
Ship
USER
AI

Microsoft 1P Team: Running AI SDLC

Multiple GA features shipped through the full agentic lifecycle

AI Shopping Advisor

RSA flagship feature, agents built end-to-end

Ratings & Reviews

Customer-facing, live in Security Store

Product Combos

Full 11-phase pipeline, zero manual coding

~60% Reduction in feature lead time vs baseline

-38% Bugs seen pre-production per feature

Agentic SDLC

An orchestrator agent owns the run end-to-end

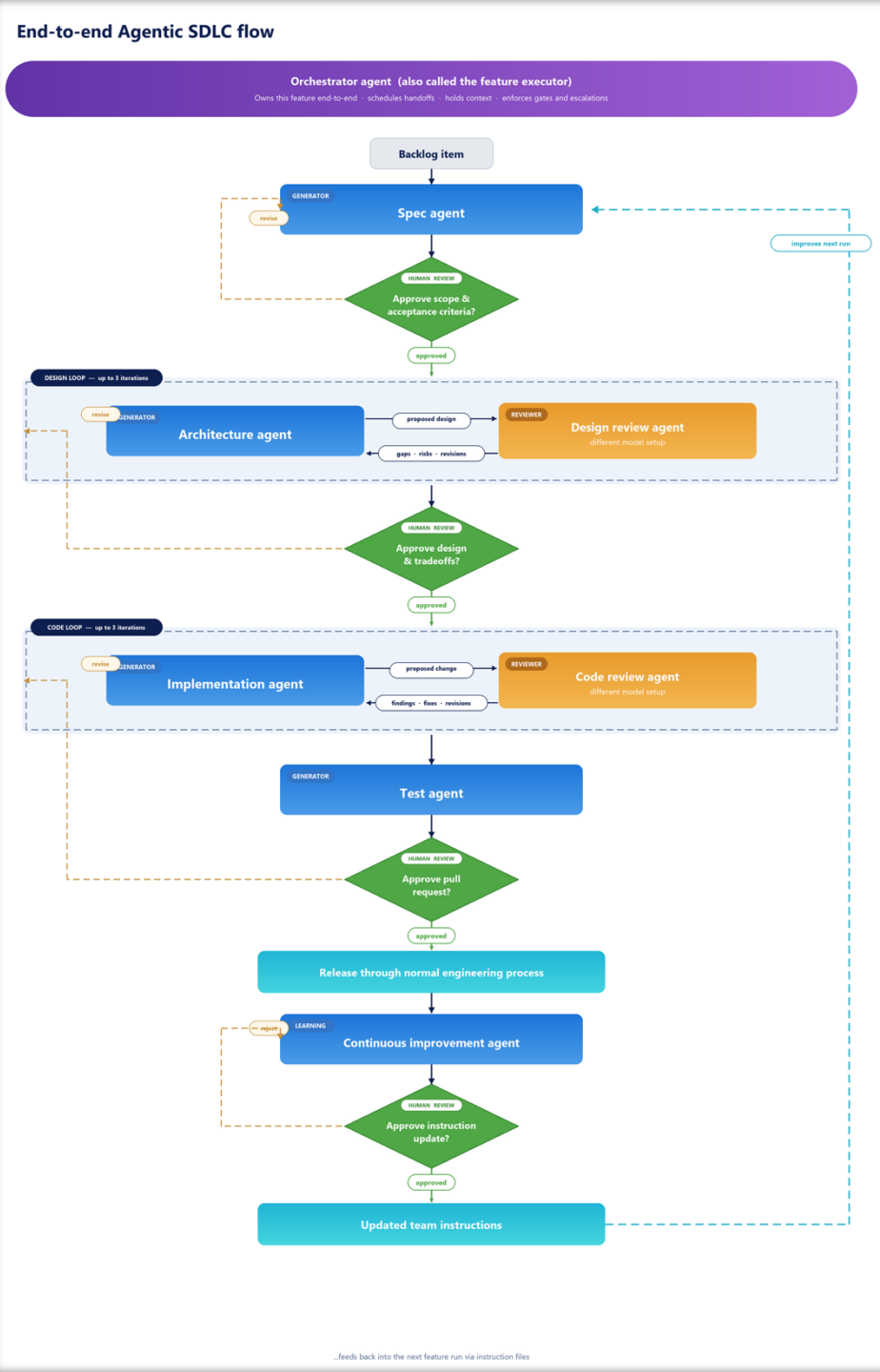
A **spec agent** turns intent into a structured requirement

An **architecture agent** proposes a design
… and a separate **design review agent** independently critiques it.

An **implementation agent** writes the code
… and a separate **code review agent** challenges it.

A **test agent** generates coverage.

A **continuous improvement agent** , a **bug hunter**, an **incident response agent**, and a **telemetry & feedback agent**



Agentic SDLC

An orchestrator agent owns the run end-to-end

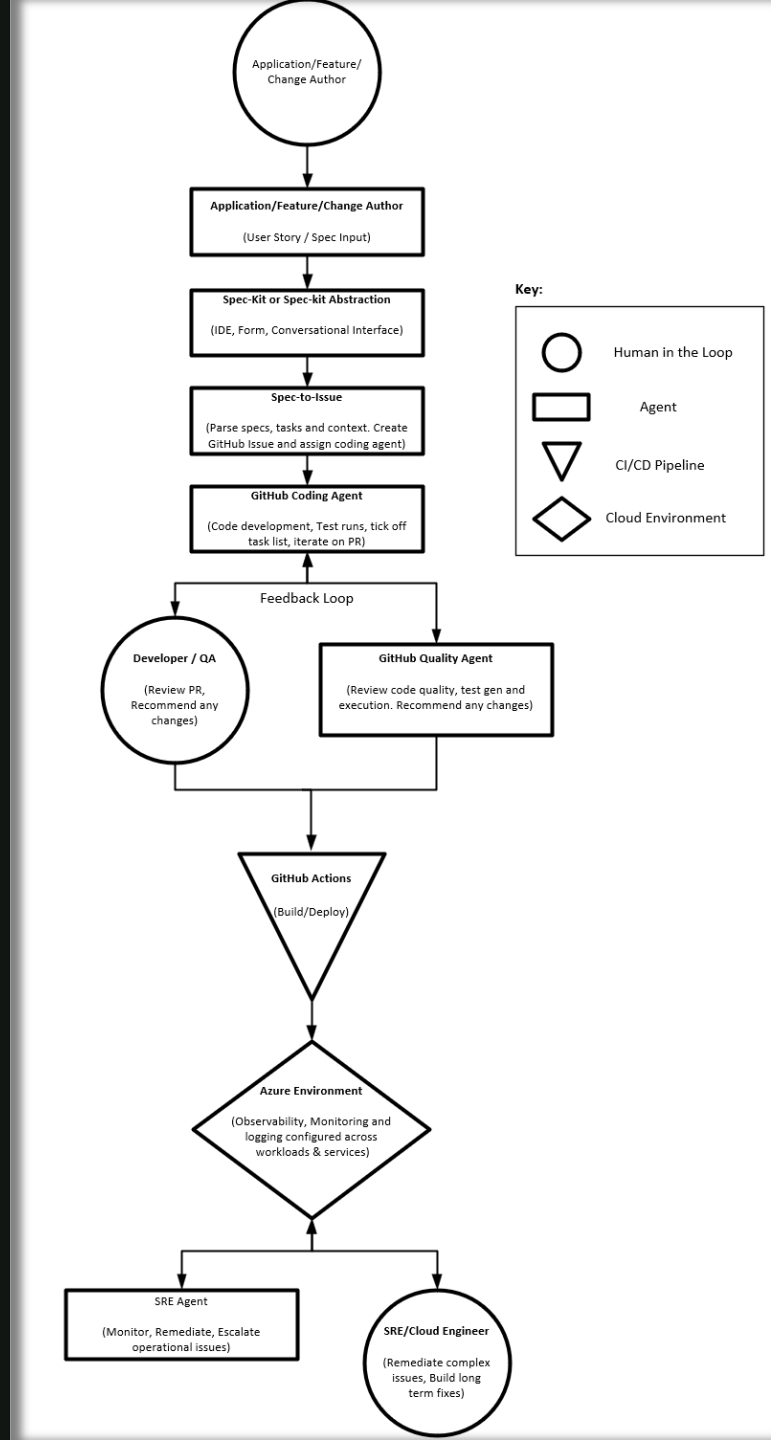
A **spec agent** turns intent into a structured requirement

An **architecture agent** proposes a design
… and a separate **design review agent** independently critiques it.

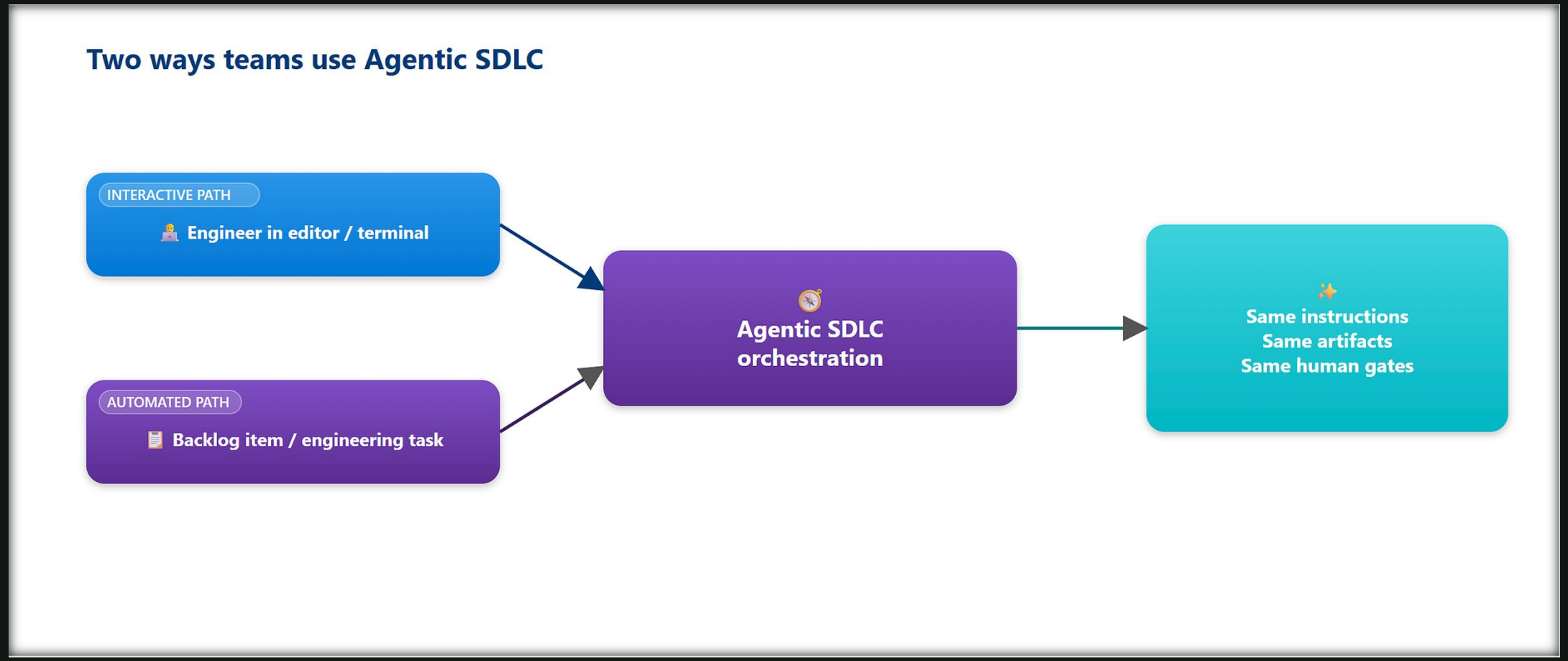
An **implementation agent** writes the code
… and a separate **code review agent** challenges it.

A **test agent** generates coverage.

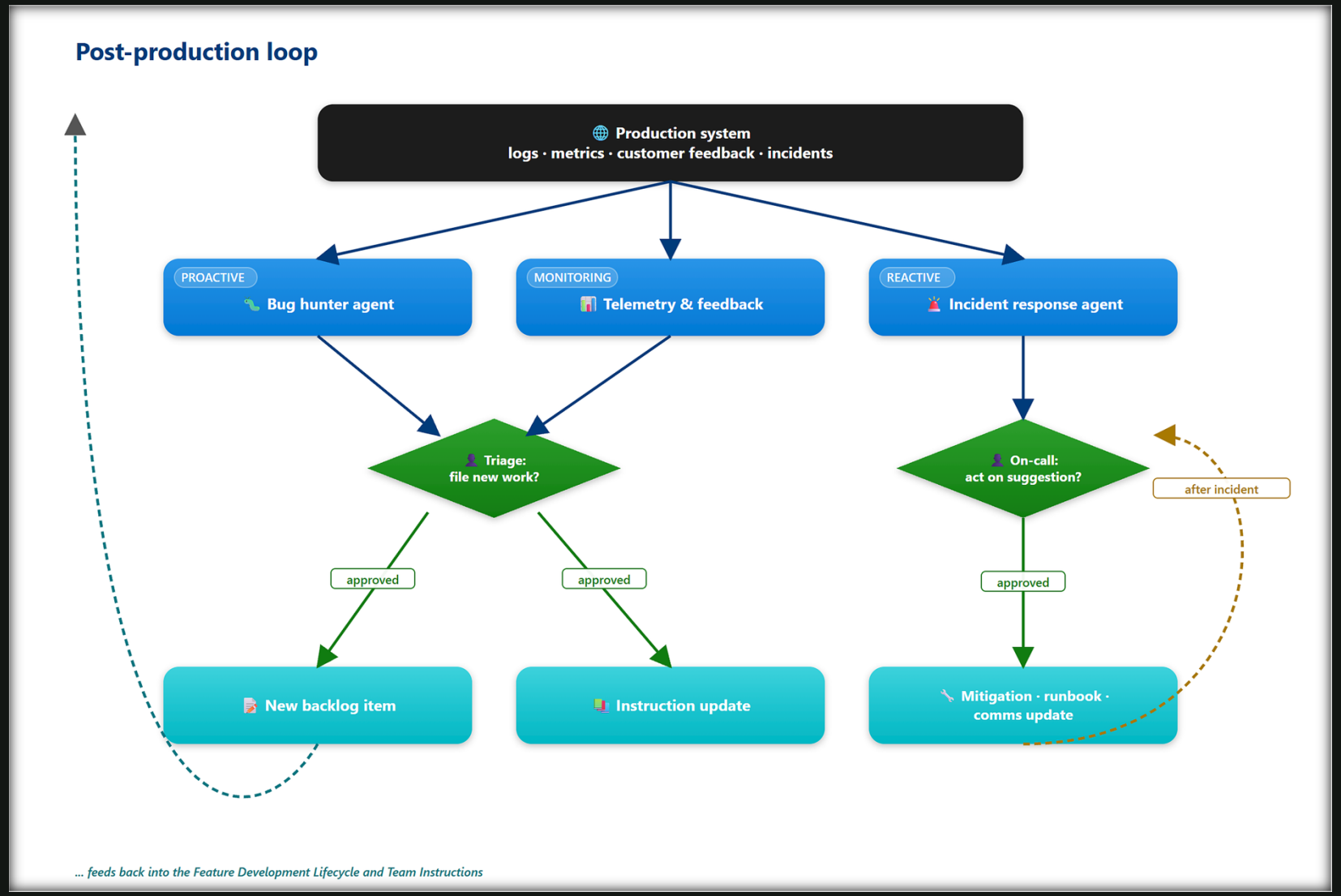
A **continuous improvement agent** , a **bug hunter**, an **incident response agent**, and a **telemetry & feedback agent**



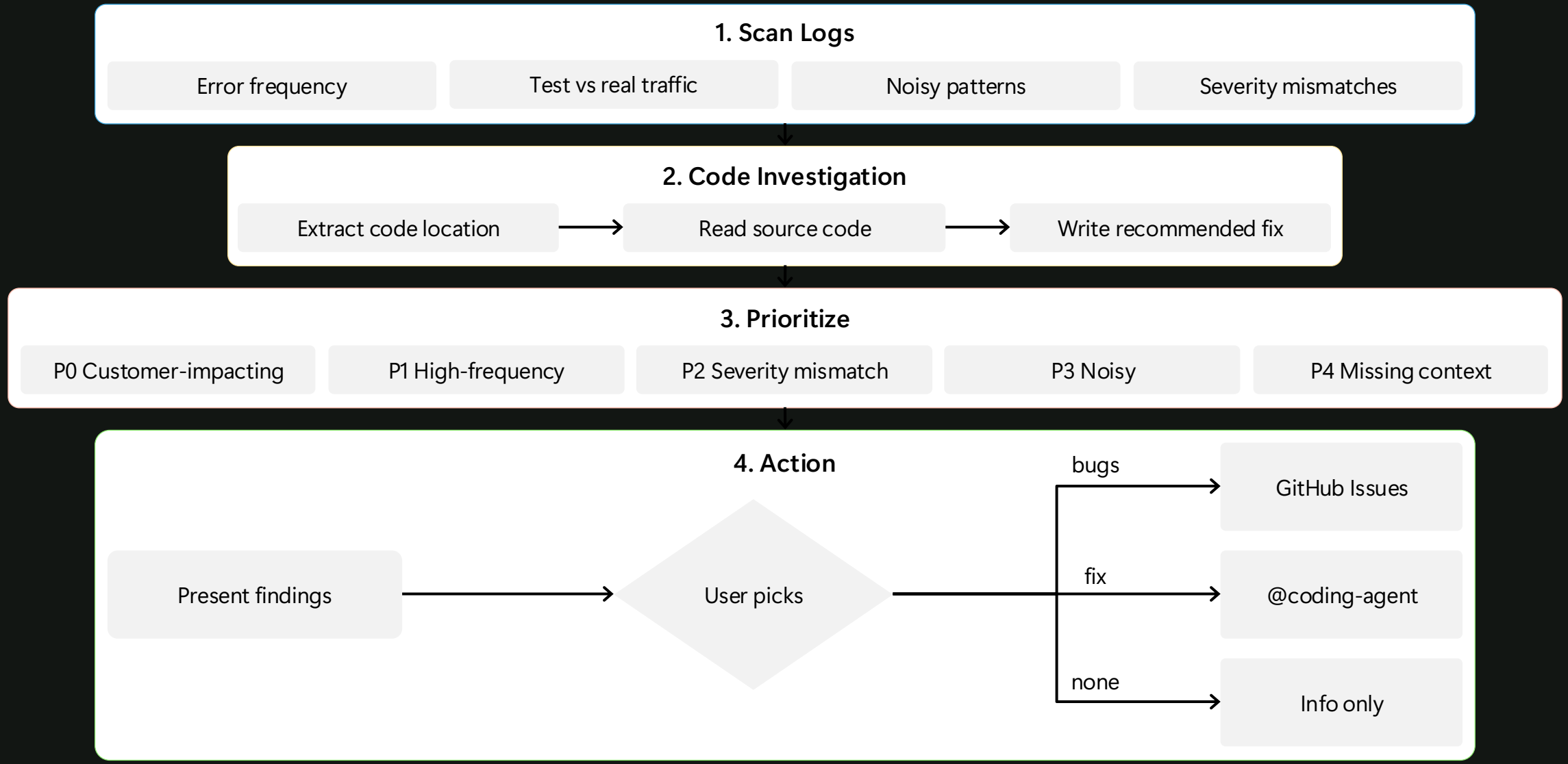
Agentic SDLC



Agentic SDLC



Self Healing System Proposal



ACROSS THE LIFECYCLE

Agents now touch every stage

Plan

Refine issues and draft specs from repo context

Code

Multi-file edits that run the build as they go

Test

Generate, run and repair tests

Review

A first-pass code review on every pull request

Secure

Suggested fixes for code scanning alerts

Operate

Triage incidents and propose fixes

> Missing from this list until recently: **keeping the stack itself current.**

WHAT MAKES IT AGENTIC

Agents close the loop

1 · Goal

Upgrade to Java 21

A clear, checkable outcome

→

2 · Plan

Write the steps

Order of changes, recipes to apply

→

3 · Act

Change and run

Edit files, run the build and tests

→

4 · Observe

Read the signals

Compile errors, failing tests, CVEs

Repeat 2 → 4 until it builds, tests pass and no known CVEs remain. Then open a pull request for a human.

> Modernization is mostly this loop: high-volume, tedious and **machine-verifiable**. That's the sweet spot for agents.

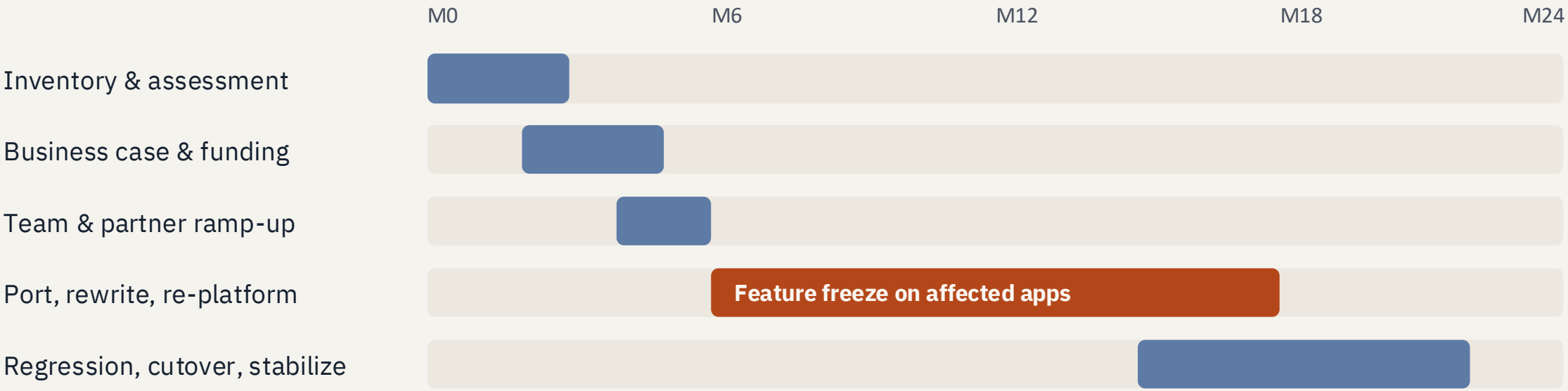
02

Modernization, the old way

A project you survive once a decade

HOW IT USED TO GO

The big-bang modernization program



> Months of planning before the first line changes, and a year or more before value ships.

Illustrative timeline for a mid-size application portfolio

THE ECONOMICS

Why it was always a one-off

Cost

Big budgets need big business cases, so it waits for a crisis.

Risk

Touching everything at once means anything can break.

Expertise

Few people know both the old stack and the new one, and they leave with the project.

Opportunity cost

Every month spent porting is a month not shipping features.

THE DRIFT CYCLE

And then the clock starts again

A big-bang project produces a snapshot. The ecosystem keeps moving:

- Java: a new release every 6 months
- Spring Boot: a new minor every 6 months
- .NET: a new major every November



03

From months to days

GitHub Copilot app modernization, end to end

MEET THE TOOL

GitHub Copilot app modernization

An agentic experience that assesses, upgrades, migrates and deploys Java and .NET applications, inside the IDE your team already uses.

Java

VS Code and IntelliJ IDEA

- Java 8+ to current LTS
- Spring Boot 2 to 3, Jakarta EE
- Migration to Azure services

.NET

Visual Studio

- .NET Framework to modern .NET
- Older .NET to current LTS
- Migration to Azure services

Under the hood

Deterministic + generative

- AppCAT for assessment
- OpenRewrite for known recipes
- The LLM for everything in between

THE WORKFLOW

One workflow, end to end

Assess

Scan code and dependencies for issues and cloud blockers

Plan

An editable upgrade plan you approve first

Upgrade

Runtime, framework and dependency changes

Validate

Build, tests, CVE scan, consistency checks

Migrate

Swap on-prem patterns for Azure services

Deploy

Containerize, provision and ship to Azure

Understand

The agent loop: change, build, test, fix

Move to the cloud

> Every step leaves a reviewable artifact: a report, a plan, a progress log, commits and a summary.

STEP 1 · ASSESS

Know what you have, in minutes

The assessment scans code, configuration and dependencies and flags:

- Outdated runtimes and frameworks
- Deprecated and removed APIs
- Known CVEs in dependencies
- Cloud-readiness blockers

Each finding links to a task the agent can run.

Finding	Category	Suggested task
Java 8 runtime	Upgrade	Upgrade to Java 21
javax.* namespaces	Upgrade	Spring Boot 3 migration
Local file storage	Cloud	Azure Blob Storage
Passwords in config	Security	Managed Identity
On-prem JMS broker	Cloud	Azure Service Bus
Vulnerable library	Security	Bump to patched version

Illustrative assessment output for a Spring Boot 2.7 app on Java 8

STEPS 2–4 · PLAN, UPGRADE, VALIDATE

The agent loop, on a real codebase

01 Drafts a plan you edit and approve

02 Applies OpenRewrite recipes

03 Builds and fixes what recipes missed

04 Runs tests and repairs regressions

05 Checks CVEs and behavior changes

06 Commits each step, writes a summary

```
[ok]   plan approved: Java 8 → 21, Boot 2.7 → 3.x
[ok]   recipes applied: [__] files changed
[fail] build: [__] compile errors
[fix]  javax → jakarta, removed APIs…
[ok]   build: success
[fail] tests: [__] failing
[fix]  repairing test config…
[ok]   tests: all passing
[ok]   CVE check: no critical findings
[ok]   summary written, ready for review
```

Illustrative run; fill in the numbers from your own demo app

STEP 5 · MIGRATE

Cloud patterns as reusable tasks

Predefined tasks

Common migrations, built in

- Passwords → Managed Identity
- Secrets → Key Vault
- Local files → Blob Storage
- JMS / RabbitMQ → Service Bus
- SMTP → Communication Services
- LDAP → Microsoft Entra ID

Custom tasks

Your patterns, captured once

Build a task from a commit that did the migration well, a markdown guide or a docs URL. Then replay it on every app that needs it.

> Tribal knowledge becomes an executable asset.

STEP 6 · DEPLOY

From modernized code to running app

Containerize

Generates Dockerfiles for each app and validates the image builds.

Provision

Infrastructure as code for Azure Container Apps, AKS or App Service.

Deploy and fix

Deploys, reads failures and logs, and iterates until it's healthy.

> Optional by design: keep your own pipeline and stop after the upgrade if you prefer.

LIVE · ~7 MIN

Demo

A Spring Boot 2.7 app on Java 8, modernized on stage.

1. Run the assessment and read the report
2. Upgrade to Java 21 and Spring Boot 3
3. Apply a custom task to a second repo
4. Hand the next upgrade to Copilot coding agent via an issue

BEFORE AND AFTER

What actually changes

	Big-bang program	Agentic modernization
Unit of work	A portfolio-wide program	One repo, one pull request
Duration	Months to years	Days to weeks
Who does it	A dedicated team plus partners	The app team, with agents
Risk	Everything changes at once	Small, reversible, committed steps
Validation	A long regression phase	Build and tests on every iteration
Knowledge	Leaves with the consultants	Stays as custom tasks

DIVISION OF LABOR

Agents do the volume. People own the judgment.

Target and sequencing

Which versions, which platform, which apps first

Review and merge

Every change lands through a human-approved pull request

Test quality

The agent is only as safe as the tests it has to pass

Rollout and risk

Canary, feature flags, when to ship to production

04

Modernization as a loop

Part of the SDLC,.

THE SHIFT

If an upgrade costs a pull request, you can do it every sprint.

Dependabot did this for libraries.
Agents extend it to runtimes, frameworks and platforms.

THE CONTINUOUS LOOP

Modernization as a stage of the SDLC

Detect
New release, CVE or EOL date

→

Assess
Agent scans affected repos

→

Transform
Agent opens a pull request

→

Validate
CI: build, tests, security

→

Merge
Owners review and ship

Observe: a fleet dashboard of runtime and framework currency feeds the next Detect.

> No program, no freeze: just a steady stream of small, reviewed changes.

WHAT YOU NEED

Building blocks you have today

Triggers

Scheduled GitHub Actions, release feeds, EOL calendars, security advisories

The agent

Copilot coding agent takes an issue and works in its own cloud environment

Know-how

Custom tasks and repo instructions encode your org's patterns

Guardrails

Branch protection, required checks, code scanning, CODEOWNERS

Scale

Assess and upgrade many repos in one batch, not one at a time

Visibility

A currency dashboard: versions, EOL dates, open modernization PRs

ILLUSTRATIVE · .GITHUB/WORKFLOWS/MODERNIZE.YML

A weekly modernization trigger

```yaml
on:
  schedule: [{ cron: "0 6 * * 1" }]  # Mondays
jobs:
  modernize:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - id: check
        run: ./scripts/check-currency.sh  # JDK, Boot, TFM
      - if: steps.check.outputs.outdated == 'true'
        run: >
          gh issue create --assignee "@copilot"
          --title "Upgrade to ${{ steps.check.outputs.target }}"
          --body-file .github/modernization/upgrade.md
```

01 · Detect

Compare versions with your policy on a schedule

02 · Delegate

The issue body is the task: target, constraints, custom task to apply

03 · Gate

Copilot opens a PR; CI and code owners decide

HOW YOU KNOW IT'S WORKING

Measure currency, not projects

Metric	Suggested target	Today
Apps on a supported runtime	100%	[__]%
Median lag behind the latest LTS	Under 1 release	[__]
Release to adoption, framework minors	Under 90 days	[__] days
Critical CVEs open more than 30 days	0	[__]
Modernization PRs merged per month	Steady, not spiky	[__]

Targets are starting points; set your own as a platform SLO

GETTING STARTED

Start small: 30, 60, 90 days

30

Prove it on one app

- Pick 2–3 representative apps
- Run assessments
- Upgrade one end to end
- Write down what the agent got wrong

60

Codify and batch

- Turn fixes into custom tasks
- Add repo instructions
- Harden CI and test gates
- Run the next 10–20 repos

90

Make it continuous

- Schedule the triggers
- Route issues to the coding agent
- Publish a currency dashboard
- Set the SLOs

RESOURCES · TRY IT THIS WEEK

Start modernizing this week

MODERNIZE CLI

Assess, plan and upgrade from the terminal, one repo or hundreds

Interactive TUI for hands-on work. Headless mode for CI/CD pipelines and batch runs across your portfolio.

```
aka.ms/ghcp-modernization-agent
```

Overview →
CLI commands →
Batch modernization →
CI/CD integration →
GitHub repository →

MCP SERVER

Put the modernization agents in any MCP host

The same assessment, upgrade and migration tools for Copilot coding agent, Copilot CLI and your own custom agents.

```
npx -y @microsoft/github-copilot-app-modernization-mcp-server
```

Overview and README →
npm package →
Custom agents →
Coding agent: Java →
Coding agent: .NET →

Documentation hub →

aka.ms/ghcp-appmod

Copilot CLI plugin →

microsoft/github-copilot-modernization

Learn training module →

Introduction to GitHub Copilot app modernization

THANK YOU · Q&A

Modernization stops being an event. It becomes a habit.



Bruno Borges · @brunoborges



Thank you

